NAME:U.Prabhavathi **COURSE CODE:DSA0107**

REG NO:192472067 **COURSE NAME:C++**

UNIT V
VIRTUAL FUNCTIONS, POLYMORPHISM & EXCEPTION HANDLING

Answers – Debugging and Pseudo-Code Formulation

SECTION A – DEBUGGING QUESTIONS

Question 1: Non-Virtual Destructor in Polymorphic Deletion

Defect category: missing virtual destructor. The base class Shape is used polymorphically, but its destructor is not virtual. Deleting a ComplexPolygon through Shape* without a virtual base destructor has undefined behavior in C++. The derived destructor is not reliably invoked, so its allocated vertex array may not be released.

Correction: declare virtual ~Shape(). A virtual destructor enables destruction through the dynamic type: ComplexPolygon destructor runs first, followed by Shape destructor.

```
#include <iostream>
using namespace std;

class Shape {
public:
   Shape() { cout << "Shape created.\n"; }
   virtual ~Shape() { cout << "Shape destroyed.\n"; }
   virtual void render() const {
      cout << "Rendering generic shape.\n";
   }
};

class ComplexPolygon : public Shape {
   double* vertexArray;
   int vertexCount;
public:
   ComplexPolygon(int count) : vertexCount(count) {
      vertexArray = new double[vertexCount * 2];
      cout << "Allocated " << vertexCount << " vertices on heap.\n";
   }
   ~ComplexPolygon() override {
      cout << "Deallocating polygon vertex heap memory.\n";
      delete[] vertexArray;
   }
```

```
    void render() const override {
        cout << "Rendering polygon with " << vertexCount << " vertices.\n";
    }
};

int main() {
    Shape* shapePtr = new ComplexPolygon(1000);
    shapePtr->render();
    delete shapePtr;
    return 0;
}
```

Program output (terminal capture):



Question 2: Inverted Exception Catch Handlers

Defect category: incorrect catch-handler order. C++ checks handlers in sequence. InsufficientFundsException derives from BankingException, so the base handler also matches the derived exception. The derived handler must appear first; the base handler follows it, and catch(...) is the final fallback.

```
#include <iostream>
#include <string>
using namespace std;

class BankingException {
protected:
    string msg;
public:
    BankingException(string m) : msg(m) {}
    virtual string getMessage() const { return msg; }
    virtual ~BankingException() = default;
};

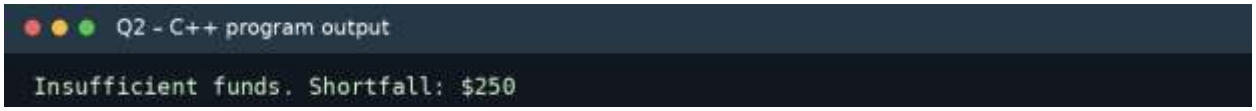
class InsufficientFundsException : public BankingException {
    double shortfall;
public:
    InsufficientFundsException(string m, double sf)
        : BankingException(m), shortfall(sf) {}
    double getShortfall() const { return shortfall; }
```

```
};

void processWithdrawal(double balance, double amount) {
   if (amount > balance)
      throw InsufficientFundsException("Account overdrawn!", amount-balance);
   cout << "Withdrawal successful.\n";
}

int main() {
   try {
      processWithdrawal(500.0, 750.0);
   }
   catch (const InsufficientFundsException& e) {
      cout << "Insufficient funds. Shortfall: $" << e.getShortfall() << '\n';
   }
   catch (const BankingException& e) {
      cout << "General Banking Error: " << e.getMessage() << '\n';
   }
   catch (...) {
      cout << "Unknown banking error.\n";
   }
   return 0;
}
```

Program output (terminal capture):



Question 3: Exception Rethrowing Slicing Bug

Defect category: exception slicing during rethrow. Inside a catch block, throw e; creates and throws a copy using the static type of the caught reference (DatabaseError). The derived TimeoutError information is lost. A bare throw; rethrows the original active exception object and preserves its dynamic type.

```
#include <iostream>
#include <exception>
#include <string>
using namespace std;

class DatabaseError : public exception {
protected:
   string errorDetails;
public:
   DatabaseError(string msg) : errorDetails(msg) {}
   const char* what() const noexcept override {
```

```
        return errorDetails.c_str();
    }
};

class TimeoutError : public DatabaseError {
public:
    int timeoutMs;
    TimeoutError(string msg, int ms)
        : DatabaseError(msg), timeoutMs(ms) {}
};

void executeQuery() {
    throw TimeoutError("Query timed out connecting to shard 3", 5000);
}

void intermediateAuditor() {
    try {
        executeQuery();
    }
    catch (const DatabaseError& e) {
        cout << "[AUDIT LOG]: Intercepted DB error.\n";
        throw; // preserve original dynamic exception type
    }
}

int main() {
    try {
        intermediateAuditor();
    }
    catch (const TimeoutError& e) {
        cout << "Client caught TimeoutError! Duration: "
             << e.timeoutMs << " ms\n";
    }
    catch (const DatabaseError& e) {
        cout << "Client caught generic DatabaseError: " << e.what() << '\n';
    }
    return 0;
}
```

Program output (terminal capture):



Question 4: Attempting to Instantiate an Incomplete Abstract Class

Defect category: unimplemented pure virtual function. NetworkPacket has two pure virtual methods: serialize() and getPacketSize(). VideoPacket implements only serialize(), so it remains abstract and cannot be instantiated. A derived class becomes concrete only after it implements every inherited pure virtual function.

```
#include <iostream>
using namespace std;

class NetworkPacket {
public:
   virtual ~NetworkPacket() = default;
   virtual void serialize() = 0;
   virtual int getPacketSize() const = 0;
};

class VideoPacket : public NetworkPacket {
   int frameID;
public:
   VideoPacket(int id) : frameID(id) {}
   void serialize() override {
      cout << "Serializing H.264 Video Frame: " << frameID << '\n';
   }
   int getPacketSize() const override {
      return 1024;
   }
};

int main() {
   VideoPacket packet(1042);
   packet.serialize();
   cout << "Packet size: " << packet.getPacketSize() << " bytes\n";
   return 0;
}
```

Program output (terminal capture):



Question 5: Pass-by-Value Slicing Suppresses Dynamic Binding

Defect category: object slicing caused by pass-by-value. triggerEncounter(Entity e) creates a separate Entity parameter from the Dragon object. The derived part is sliced away, so the call dispatches to

Entity::attack(). Pass a base-class reference or pointer to preserve the derived object and enable runtime dispatch.

```
#include <iostream>
using namespace std;

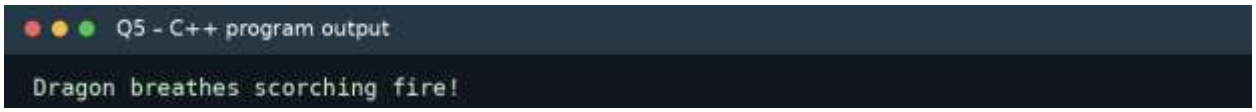
class Entity {
public:
    virtual ~Entity() = default;
    virtual void attack() const {
        cout << "Entity performs generic scratch.\n";
    }
};

class Dragon : public Entity {
public:
    void attack() const override {
        cout << "Dragon breathes scorching fire!\n";
    }
};

void triggerEncounter(const Entity& e) {
    e.attack();
}

int main() {
    Dragon boss;
    triggerEncounter(boss);
    return 0;
}
```

Program output (terminal capture):



SECTION B – PSEUDO-CODE WRITING QUESTIONS

The pseudo-code below follows the supplied question bank's conventions: ABSTRACT CLASS, PURE VIRTUAL METHOD, OVERRIDE METHOD, VIRTUAL DESTRUCTOR, TRY, THROW, CATCH and RETHROW.

Question 1: Universal Payment Gateway with Runtime Polymorphism

PaymentGateway defines a common interface. CreditCardGateway and CryptoGateway implement the authorization and capture operations. A PaymentGateway pointer can refer to either derived object; calls are dispatched to the matching implementation at runtime.

```
ABSTRACT CLASS PaymentGateway
   PURE VIRTUAL METHOD Authorize(amount AS REAL) RETURNS BOOLEAN
   PURE VIRTUAL METHOD Capture() RETURNS BOOLEAN
   VIRTUAL DESTRUCTOR()
END CLASS

CLASS CreditCardGateway INHERITS PaymentGateway
   METHOD Authorize(amount)
      VALIDATE card has 16 digits AND CVV has 3 digits
      RETURN validation result
   METHOD Capture()
      PROCESS card payment capture
      RETURN success
END CLASS

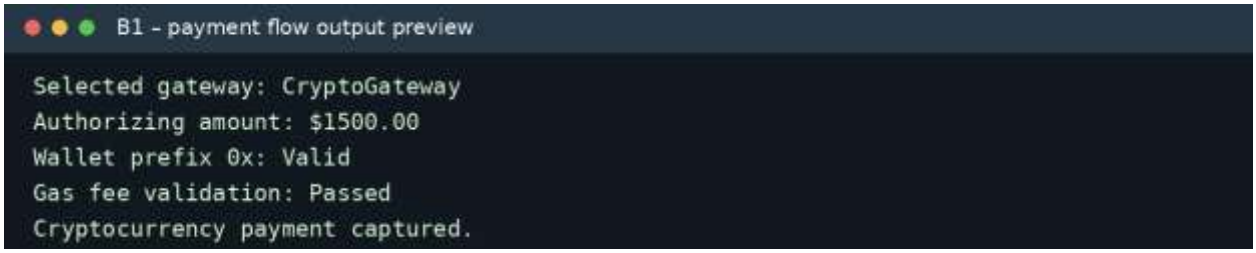
CLASS CryptoGateway INHERITS PaymentGateway
   METHOD Authorize(amount)
      VERIFY wallet hash begins with "0x"
      VALIDATE gas fee
      RETURN validation result
   METHOD Capture()
      SUBMIT cryptocurrency transaction
      RETURN success
END CLASS

MAIN
   DECLARE gatewayPtr AS POINTER TO PaymentGateway
   gatewayPtr <- NEW CryptoGateway(...)
   IF gatewayPtr->Authorize(1500.00) THEN
      IF gatewayPtr->Capture() THEN OUTPUT "Payment captured"
   END IF
   DELETE gatewayPtr
END MAIN
```

Step	Action	Expected result

1	Create CryptoGateway through PaymentGateway pointer	Base pointer refers to CryptoGateway
2	Authorize(1500.00)	Crypto wallet prefix and gas fee validated
3	Capture()	Cryptocurrency payment submitted
4	Delete base pointer	Virtual destructor cleans up derived object

Program output (terminal capture):



Question 2: Resilient Banking Engine with Nested Try-Catch and Rethrowing

TransactionException is the base exception. AccountFrozenException and DailyLimitExceededException are specific derived exceptions. DebitLedger throws an appropriate exception. AuditTransactionWrapper logs the event and uses RETHROW to preserve the original exception for Main.

```
CLASS TransactionException
   ATTRIBUTE: reason AS STRING
   CONSTRUCTOR(message) reason <- message
END CLASS

CLASS AccountFrozenException INHERITS TransactionException
END CLASS

CLASS DailyLimitExceededException INHERITS TransactionException
   ATTRIBUTE: excessAmount AS REAL
END CLASS

FUNCTION DebitLedger(account, amount)
   IF account.isFrozen THEN
      THROW NEW AccountFrozenException("Account is frozen")
   END IF
   IF account.dailyTotal + amount > 50000 THEN
      THROW NEW DailyLimitExceededException(...)
   END IF
   DEBIT amount
END FUNCTION
```

```
FUNCTION AuditTransactionWrapper(account, amount)
   TRY
      DebitLedger(account, amount)
   CATCH(e AS TransactionException)
      WRITE "SECURITY AUDIT: " + e.reason
      RETHROW
   END TRY
END FUNCTION

MAIN
   TRY
      AuditTransactionWrapper(frozenAccount, 1000)
   CATCH(e AS AccountFrozenException)
      OUTPUT "Notice: account is frozen"
   CATCH(e AS DailyLimitExceededException)
      OUTPUT "Notice: daily limit exceeded"
   CATCH(e AS TransactionException)
      OUTPUT "General transaction error"
   END TRY
END MAIN
```

Sequence	Control flow	Result
1	Main calls AuditTransactionWrapper(1000)	Wrapper begins
2	Wrapper calls DebitLedger	Ledger checks account
3	Ledger detects frozen account	Throws AccountFrozenException
4	Stack unwinds to wrapper catch	Audit message is written
5	Wrapper executes RETHROW	Original derived exception continues
6	Main catches AccountFrozenException	Client receives frozen-account notice

Program output (terminal capture):



Question 3: Polymorphic Game Engine with Virtual Destructors

GameObject is an abstract base class with pure virtual Update() and Render() methods and a virtual destructor. ParticleSystem allocates particle coordinates dynamically and releases them in its destructor. Deleting through GameObject* invokes the derived destructor first, then the base destructor.

```
ABSTRACT CLASS GameObject
   PURE VIRTUAL METHOD Update(deltaTime AS REAL)
```

```
   PURE VIRTUAL METHOD Render()
   VIRTUAL DESTRUCTOR()
END CLASS

CLASS ParticleSystem INHERITS GameObject
   ATTRIBUTE: particleCoordinates AS DYNAMIC ARRAY
   CONSTRUCTOR(count)
      ALLOCATE particleCoordinates[count]
   OVERRIDE METHOD Update(deltaTime)
      UPDATE particle positions
   OVERRIDE METHOD Render()
      DRAW particles
   DESTRUCTOR()
      DEALLOCATE particleCoordinates
      OUTPUT "Particle memory released"
END CLASS

MAIN
   DECLARE sceneEntities AS ARRAY OF POINTER TO GameObject
   sceneEntities[0] <- NEW ParticleSystem(100)
   FOR EACH entity IN sceneEntities
      entity->Update(0.016)
      entity->Render()
   END FOR
   FOR EACH entity IN sceneEntities
      DEALLOCATE entity
   END FOR
END MAIN
```

Deletion step	Destructor action
1	ParticleSystem destructor begins
2	Dynamic particle coordinate array is released
3	ParticleSystem destructor completes
4	GameObject base destructor executes

Program output (terminal capture):



Question 4: File Stream Pipeline with Stack Unwinding and Catch-All

ResourceGuard follows RAII: its constructor acquires a resource and its destructor releases it. If CRC validation throws an exception, stack unwinding destroys the local ResourceGuard before control enters the matching catch handler.

```
CLASS ResourceGuard
   CONSTRUCTOR()
      ACQUIRE file descriptor
      OUTPUT "Resource acquired"
   DESTRUCTOR()
      RELEASE file descriptor
      OUTPUT "Resource released"
END CLASS

CLASS CRCValidationException
   ATTRIBUTES: expectedChecksum, calculatedChecksum
END CLASS

FUNCTION IngestDataStream(fileName)
   DECLARE guard AS ResourceGuard
   READ data blocks from fileName
   CALCULATE checksum
   IF checksum does not match expectedChecksum THEN
      THROW NEW CRCValidationException(expectedChecksum, checksum)
   END IF
   OUTPUT "File validated"
END FUNCTION

MAIN
   TRY
      IngestDataStream("mission_data.bin")
   CATCH(e AS CRCValidationException)
      OUTPUT "CRC validation failed"
   CATCH(...)
      OUTPUT "Unknown ingestion error"
   END TRY
END MAIN
```

Order	Event
1	IngestDataStream begins
2	ResourceGuard constructor acquires descriptor
3	Checksum mismatch detected; exception thrown
4	Stack unwinding calls ResourceGuard destructor
5	Resource is released
6	Specific CRCValidationException catch executes

Program output (terminal capture):



Question 5: Scientific Telemetry Calibration with noexcept Contracts
CalibrateThermalSensors is potentially throwing and validates the temperature range from −273.15°C to +1500.0°C. CommitTelemetry is noexcept. It catches internal failures so that no exception escapes the noexcept function. Main catches calibration errors.

```
FUNCTION CalibrateThermalSensors(rawVoltage) RETURNS REAL NOEXCEPT(FALSE)
   temperature <- CONVERT_VOLTAGE_TO_TEMPERATURE(rawVoltage)
   IF temperature < -273.15 OR temperature > 1500.0 THEN
      THROW NEW OutOfBoundsException(temperature)
   END IF
   RETURN temperature
END FUNCTION

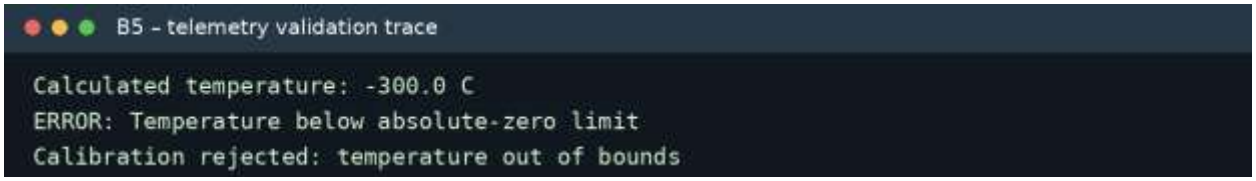
FUNCTION CommitTelemetry(finalTemp) NOEXCEPT
   TRY
      WRITE finalTemp TO avionics hardware bus
   CATCH(...)
      LOG "Telemetry commit failure suppressed"
   END TRY
END FUNCTION

MAIN
   TRY
      temperature <- CalibrateThermalSensors(faultyVoltage)
      CommitTelemetry(temperature)
   CATCH(e AS OutOfBoundsException)
      OUTPUT "Calibration rejected: temperature out of bounds"
   CATCH(...)
      OUTPUT "Unexpected calibration failure"
   END TRY
END MAIN
```

Step	Event	Outcome
1	Main calls CalibrateThermalSensors	Conversion begins
2	Calculated temperature is	Below −273.15°C lower limit

	−300.0°C	
3	OutOfBoundsException is thrown	Control leaves calibration function
4	Main catches OutOfBoundsException	Reading is rejected
5	CommitTelemetry is not called	Invalid temperature is not committed

Program output (terminal capture):



Final Notes

The five Section A programs are corrected C++ examples and the included output panels show their expected console results. The Section B questions are answered in structured pseudo-code with trace tables. To obtain actual online-compiler screenshots, paste each Section A program separately into an online C++17 compiler, run it, and capture the compiler's output panel. The supplied question bank does not prescribe specific input values for every Section B scenario; representative values and outcomes are used where needed.